Lab 2B: Masking Data with Column Masks and Row Filters

Description

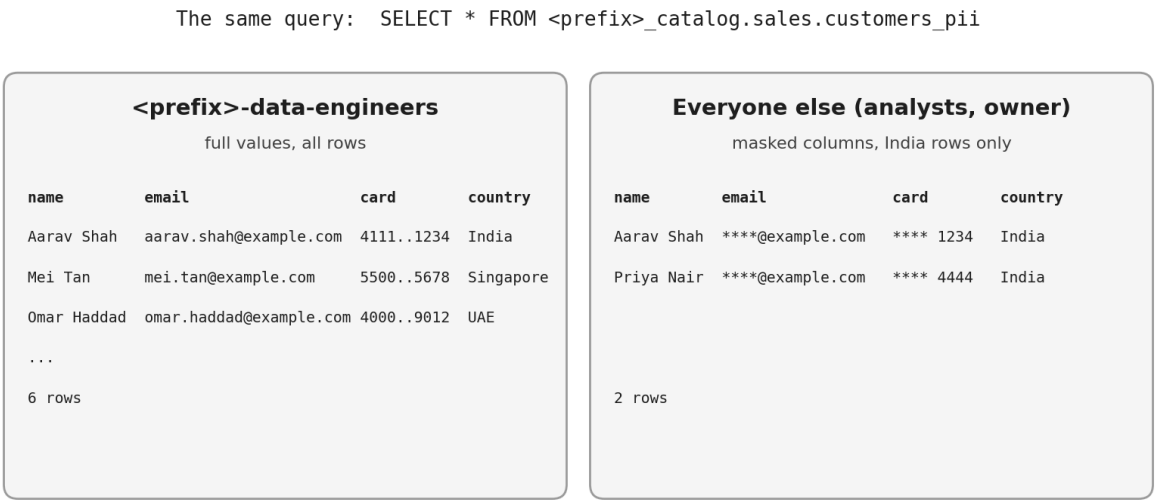
You will create a table of customer personal data in your Lab 2 catalog, then protect it with Unity Catalog column masks and a row filter. Everyone runs the same query, but Data Engineers see full values and all rows, while Data Analysts see masked values and only India's customers.

Learning Objectives

- Write SQL functions that decide what each group can see
- Apply column masks to sensitive columns
- Apply a row filter to limit which rows are returned
- Verify the result as different users with the same query

Step by Step Guide

What you will build



Your lab values

Item	Value
Catalog and schema	<prefix>_catalog.sales (from Lab 2)
New table	customers_pii
Sees full data	<prefix>-data-engineers
Sees masked data, India only	Everyone else, including you as the owner
Test users	<prefix>-engineer1, <prefix>-analyst1
Compute	Shared serverless SQL warehouse, in SQL Editor

Tip: As in Lab 2, paste each script into SQL Editor and use Find and Replace to change every <prefix>.

Step 1: Create the table with personal data

Sign in with your learner account, open SQL Editor on the shared warehouse and run:

```
CREATE TABLE IF NOT EXISTS <prefix>_catalog.sales.customers_pii (
  customer_id INT, name STRING, email STRING,
  phone STRING, card_number STRING, country STRING);

INSERT INTO <prefix>_catalog.sales.customers_pii VALUES
  (1, 'Aarav Shah', 'aarav.shah@example.com', '+91 98100 12345',
     '4111 1111 1111 1234', 'India'),
  (2, 'Mei Tan', 'mei.tan@example.com', '+65 8123 4567',
     '5500 0000 0000 5678', 'Singapore'),
  (3, 'Omar Haddad', 'omar.haddad@example.com', '+971 50 123 4567',
     '4000 0000 0000 9012', 'UAE'),
  (4, 'Grace Wanjiru', 'grace.w@example.com', '+254 712 345678',
     '3400 0000 0000 3456', 'Kenya'),
  (5, 'Liam Walker', 'liam.walker@example.com', '+61 412 345 678',
     '6011 0000 0000 7890', 'Australia'),
  (6, 'Priya Nair', 'priya.nair@example.com', '+91 99000 54321',
     '4111 2222 3333 4444', 'India');

SELECT * FROM <prefix>_catalog.sales.customers_pii;
```

> Check: You see 6 rows with full emails, phones and card numbers. All values are made up.

Step 2: Create the masking functions

Each function returns the real value for members of your Data Engineers group, and a masked value for everyone else.

```
CREATE OR REPLACE FUNCTION <prefix>_catalog.sales.mask_email(email STRING)
RETURNS STRING
RETURN CASE WHEN is_account_group_member('<prefix>-data-engineers') THEN email
            ELSE concat('****@', split_part(email, '@', 2)) END;

CREATE OR REPLACE FUNCTION <prefix>_catalog.sales.mask_phone(phone STRING)
RETURNS STRING
RETURN CASE WHEN is_account_group_member('<prefix>-data-engineers') THEN phone
            ELSE regexp_replace(phone, '[0-9]', '*') END;

CREATE OR REPLACE FUNCTION <prefix>_catalog.sales.mask_card(card STRING)
RETURNS STRING
RETURN CASE WHEN is_account_group_member('<prefix>-data-engineers') THEN card
            ELSE concat('**** ', right(replace(card, ' ', ''), 4)) END;
```

> Check: Three functions appear under sales > Functions in Catalog.

Step 3: Apply the column masks

```
ALTER TABLE <prefix>_catalog.sales.customers_pii
  ALTER COLUMN email SET MASK <prefix>_catalog.sales.mask_email;
ALTER TABLE <prefix>_catalog.sales.customers_pii
  ALTER COLUMN phone SET MASK <prefix>_catalog.sales.mask_phone;
ALTER TABLE <prefix>_catalog.sales.customers_pii
  ALTER COLUMN card_number SET MASK <prefix>_catalog.sales.mask_card;

SELECT * FROM <prefix>_catalog.sales.customers_pii;
```

> Check: Even you, the owner, now see masked values, because you are not in the Data Engineers group. Masks apply to everyone who queries the table.

Step 4: Create and apply the row filter

The filter returns TRUE for rows a user may see: every row for Data Engineers, and only India's rows for everyone else.

```
CREATE OR REPLACE FUNCTION <prefix>_catalog.sales.country_filter(country STRING)
RETURNS BOOLEAN
RETURN is_account_group_member('<prefix>-data-engineers') OR country = 'India';

ALTER TABLE <prefix>_catalog.sales.customers_pii
  SET ROW FILTER <prefix>_catalog.sales.country_filter ON (country);

SELECT * FROM <prefix>_catalog.sales.customers_pii;
```

> Check: You now see only 2 rows, Aarav and Priya, with masked values.

Step 5: Check the table's protection

```
DESCRIBE TABLE EXTENDED <prefix>_catalog.sales.customers_pii;
```

> Check: Scroll through the output to find the row filter and the column masks with their function names.

Note: No new grants are needed. In Lab 2 you granted SELECT on the whole sales schema to both groups, so they can already query the new table. Masks and filters decide what they see.

Step 6: Verify as the Data Engineer

1. Open a private browser window and sign in as <prefix>-engineer1@saurabhuptut.onmicrosoft.com.
2. Open SQL Editor, select the shared warehouse and run:

```
SELECT * FROM <prefix>_catalog.sales.customers_pii;
SELECT count(*) FROM <prefix>_catalog.sales.customers_pii;
```

> Check: The engineer sees all 6 rows with full emails, phones and card numbers.

Step 7: Verify as the Data Analyst

1. Open a new private window and sign in as <prefix>-analyst1@saurabhuptut.onmicrosoft.com.
2. Open SQL Editor and run the same queries, plus a search on email:

```
SELECT * FROM <prefix>_catalog.sales.customers_pii;
SELECT count(*) FROM <prefix>_catalog.sales.customers_pii;
SELECT * FROM <prefix>_catalog.sales.customers_pii WHERE email LIKE 'aarav%';
```

> Check: The analyst sees 2 rows, count is 2, and values are masked. The email search returns nothing, because the analyst only ever sees the masked email.

Step 8 (Optional): Remove the protection and re-test

As the owner, run:

```
ALTER TABLE <prefix>_catalog.sales.customers_pii DROP ROW FILTER;
ALTER TABLE <prefix>_catalog.sales.customers_pii ALTER COLUMN email DROP MASK;
```

1. As <prefix>-analyst1, run SELECT * again. All 6 rows appear, emails are visible, and phone and card are still masked.
2. As the owner, apply the row filter and email mask again using the statements from Steps 3 and 4.

> Check: Protection changes take effect on the next query, with no change to the data or to grants.

Outcomes

- customers_pii is protected by three column masks and one row filter
- Data Engineers see full values and all rows; everyone else sees masked values and India only
- You saw that masks and filters apply to the owner too
- Access to the table and what users see inside it are controlled separately